

W4111 – Introduction to Databases
Section 002, Fall 2025
Lecture 8: Foundation – ER, Relational, SQL (4)
REST, Normalization



Introduction

Contents

Content

Course Overview and Discussion

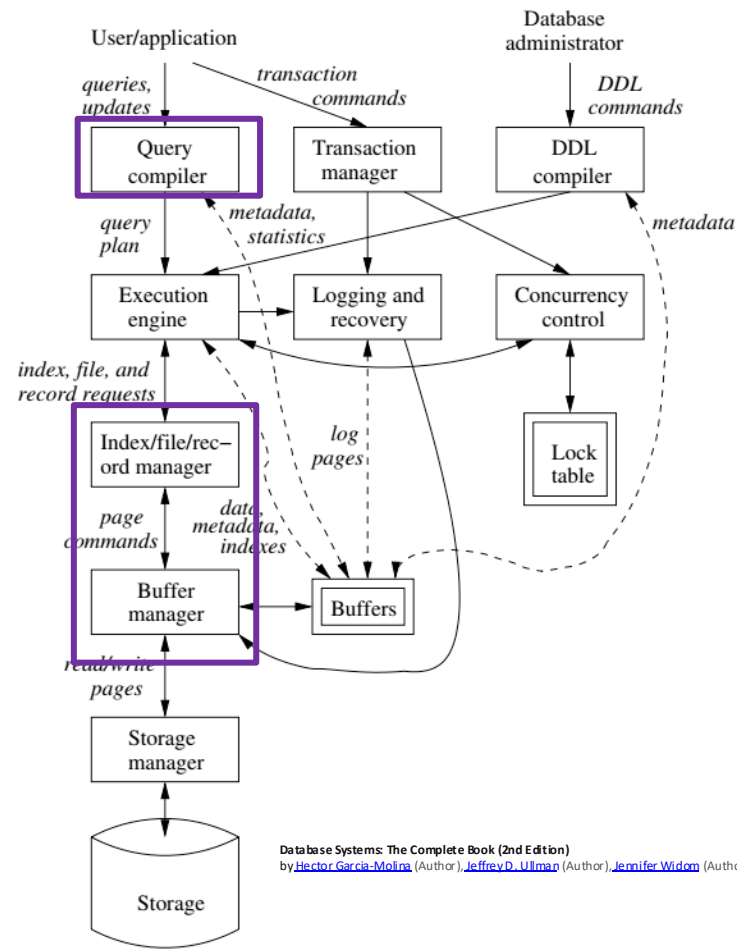
Course Overview and Discussion

Module II

Introduction and Reminder

Today

- Find things quickly.
- Query things quickly and efficiently.

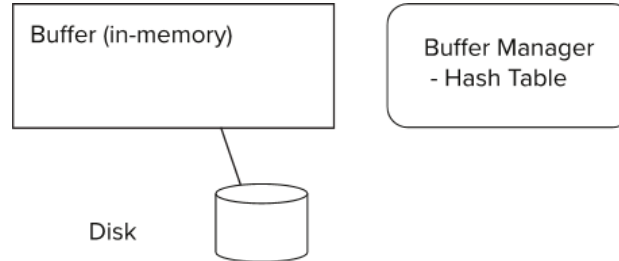


Buffer Pool/Buffer Manager



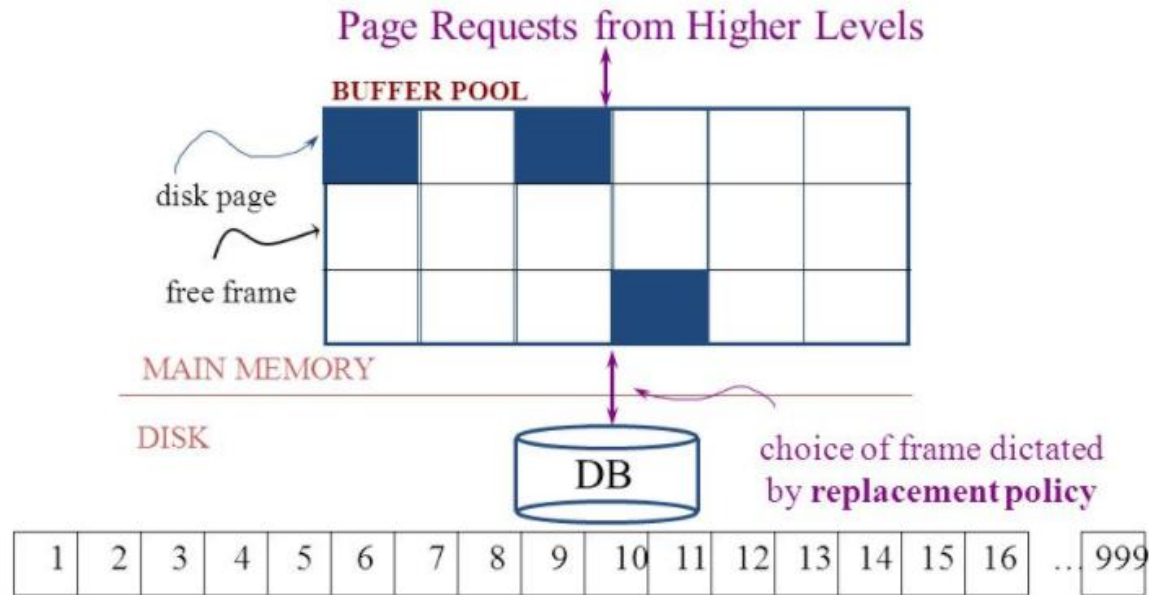
Storage Access

- Blocks are units of both storage allocation and data transfer.
- Database system seeks to minimize the number of block transfers between the disk and memory. We can reduce the number of disk accesses by keeping as many blocks as possible in main memory.
- **Buffer** – portion of main memory available to store copies of disk blocks.
- **Buffer manager** – subsystem responsible for allocating buffer space in main memory.



The Logical Concept

- The DBMS and queries can only manipulate in-memory blocks and records.
- A very, very, very small fraction of all blocks fit in memory.





Buffer Manager

- Programs call on the buffer manager when they need a block from disk.
 - If the block is already in the buffer, buffer manager returns the address of the block in main memory
 - If the block is not in the buffer, the buffer manager
 - Allocates space in the buffer for the block
 - Replacing (throwing out) some other block, if required, to make space for the new block.
 - Replaced block written back to disk only if it was modified since the most recent time that it was written to/fetched from the disk.
 - Reads the block from the disk to the buffer, and returns the address of the block in main memory to requester.



Buffer Manager

- **Buffer replacement strategy** (details coming up!)
- **Pinned block:** memory block that is not allowed to be written back to disk
 - **Pin** done before reading/writing data from a block
 - **Unpin** done when read /write is complete
 - Multiple concurrent pin/unpin operations possible
 - Keep a pin count, buffer block can be evicted only if pin count = 0
- **Shared and exclusive locks on buffer**
 - Needed to prevent concurrent operations from reading page contents as they are moved/reorganized, and to ensure only one move/reorganize at a time
 - Readers get shared lock, updates to a block require exclusive lock
 - **Locking rules:**
 - Only one process can get exclusive lock at a time
 - Shared lock cannot be concurrently with exclusive lock
 - Multiple processes may be given shared lock concurrently



Buffer-Replacement Policies

- Most operating systems replace the block **least recently used** (LRU strategy)
 - Idea behind LRU – use past pattern of block references as a predictor of future references
 - LRU can be bad for some queries
- Queries have well-defined access patterns (such as sequential scans), and a database system can use the information in a user's query to predict future references
- Mixed strategy with hints on replacement strategy provided by the query optimizer is preferable
- Example of bad access pattern for LRU: when computing the join of 2 relations r and s by a nested loops
 - for each tuple tr of r do
 - for each tuple ts of s do
 - if the tuples tr and ts match ...



Buffer-Replacement Policies (Cont.)

- **Toss-immediate** strategy – frees the space occupied by a block as soon as the final tuple of that block has been processed
- **Most recently used (MRU) strategy** – system must pin the block currently being processed. After the final tuple of that block has been processed, the block is unpinned, and it becomes the most recently used block.
- Buffer manager can use statistical information regarding the probability that a request will reference a particular relation
 - E.g., the data dictionary is frequently accessed. Heuristic: keep data-dictionary blocks in main memory buffer
- Operating system or buffer manager may reorder writes
 - Can lead to corruption of data structures on disk
 - E.g., linked list of blocks with missing block on disk
 - File systems perform consistency check to detect such situations
 - Careful ordering of writes can avoid many such problems

Replacement Policy

- The *replacement policy* is one of the most important factors in database management system implementation and configuration.
- A very simple, introductory explanation is (https://en.wikipedia.org/wiki/Cache_replacement_policies).
 - There are a lot of possible policies.
 - The *most* efficient caching algorithm would be to always discard the information that will not be needed for the longest time in the future. This optimal result is referred to as [Bélády's](#) optimal algorithm/simplely optimal replacement policy or [the clairvoyant algorithm](#).
- All implementable policies are an attempt to approximate knowledge of the future based on knowledge of the past.
- Least Recently Used is based on the simplest assumption
 - The information that will not be needed for the longest time.
 - Is the information that has not been accessed for the longest time.

Replacement Algorithm

The algorithms are more sophisticated in the real world, e.g.

- “Scans” are common, e.g. go through a large query result in order (will be more clear when discussing cursors).
 - The engine knows the current position in the result set.
 - Uses the sort order to determine which records will be accessed soon.
 - Tags those blocks as not replaceable.
 - (A form of clairvoyance).
- Not all users/applications are equally “important.”
 - Classify users/applications into priority 1, 2 and 3.
 - Sub-allocate the buffer pool into pools P1, P2 and P3.
 - Apply LRU within pools and adjust pool sizes based on relative importance.
 - This prevents
 - A high access rate, low-priority application from taking up a lot of frames
 - Result in low access, high priority applications not getting buffer hits.



Optimization of Disk Block Access (Cont.)

- Buffer managers support **forced output** of blocks for the purpose of recovery (more in Chapter 19)
- **Nonvolatile write buffers** speed up disk writes by writing blocks to a non-volatile RAM or flash buffer immediately
 - *Writes can be reordered to minimize disk arm movement*
- **Log disk** – a disk devoted to writing a sequential log of block updates
 - Used exactly like nonvolatile RAM
 - Write to log disk is very fast since no seeks are required
- **Journaling file systems** write data in-order to NV-RAM or log disk
 - Reordering without journaling: risk of corruption of file system data

Indexes



Basic Concepts

- Indexing mechanisms used to speed up access to desired data.
 - E.g., author catalog in library
- **Search Key** - attribute to set of attributes used to look up records in a file.
- An **index file** consists of records (called **index entries**) of the form

search-key	pointer
------------	---------

- Index files are typically much smaller than the original file
- Two basic kinds of indices:
 - **Ordered indices:** search keys are stored in sorted order
 - **Hash indices:** search keys are distributed uniformly across “buckets” using a “hash function”.

(“c:\”, LBN, offset)

search-key	pointer
search-key	pointer
search-key	pointer



Ordered Indices

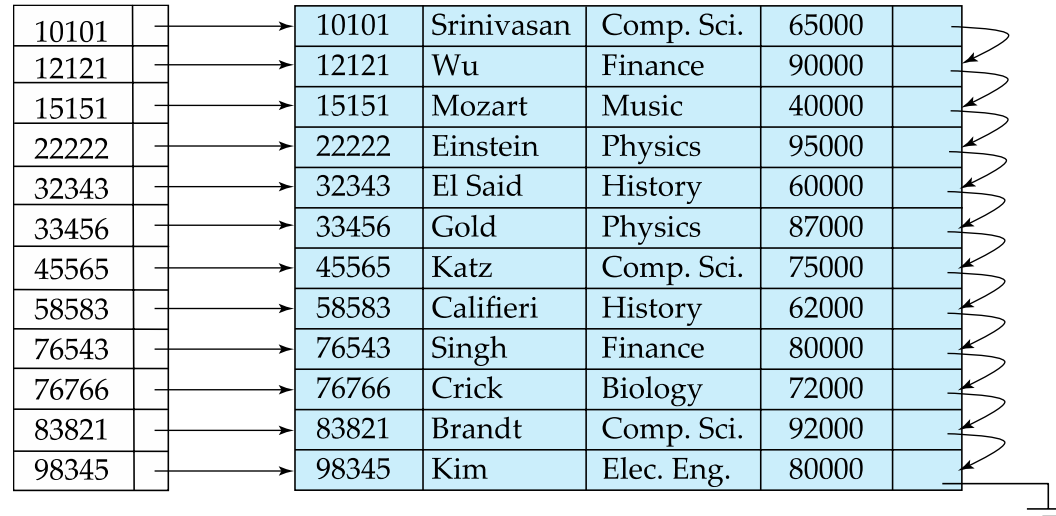
- In an **ordered index**, index entries are stored sorted on the search key value.
- **Clustering index**: in a sequentially ordered file, the index whose search key specifies the sequential order of the file.
 - Also called **primary index**
 - The search key of a primary index is usually but not necessarily the primary key.
- **Secondary index**: an index whose search key specifies an order different from the sequential order of the file. Also called **nonclustering index**.
- **Index-sequential file**: sequential file ordered on a search key, with a clustering index on the search key.



Dense Index Files

- **Dense index** — Index record appears for every search-key value in the file.
- E.g. index on *ID* attribute of *instructor* relation

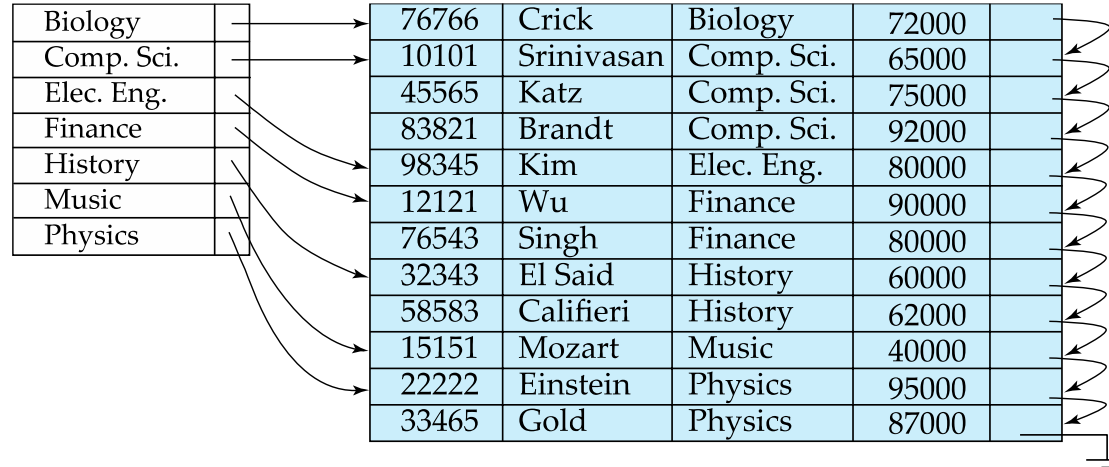
10101		→	10101	Srinivasan	Comp. Sci.	65000	
12121		→	12121	Wu	Finance	90000	
15151		→	15151	Mozart	Music	40000	
22222		→	22222	Einstein	Physics	95000	
32343		→	32343	El Said	History	60000	
33456		→	33456	Gold	Physics	87000	
45565		→	45565	Katz	Comp. Sci.	75000	
58583		→	58583	Califieri	History	62000	
76543		→	76543	Singh	Finance	80000	
76766		→	76766	Crick	Biology	72000	
83821		→	83821	Brandt	Comp. Sci.	92000	
98345		→	98345	Kim	Elec. Eng.	80000	





Dense Index Files (Cont.)

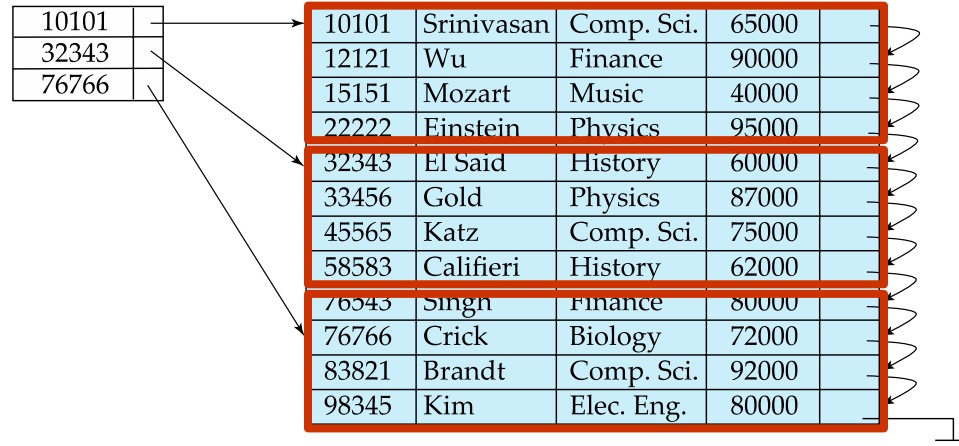
- Dense index on *dept_name*, with *instructor* file sorted on *dept_name*





Sparse Index Files

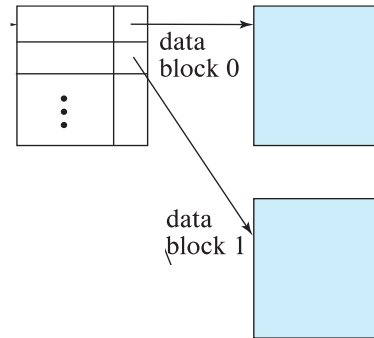
- **Sparse Index:** contains index records for only some search-key values.
 - Applicable when records are sequentially ordered on search-key
- To locate a record with search-key value K we:
 - Find index record with largest search-key value $< K$
 - Search file sequentially starting at the record to which the index record points





Sparse Index Files (Cont.)

- Compared to dense indices:
 - Less space and less maintenance overhead for insertions and deletions.
 - Generally slower than dense index for locating records.
- **Good tradeoff:**
 - for clustered index: sparse index with an index entry for every block in file, corresponding to least search-key value in the block.

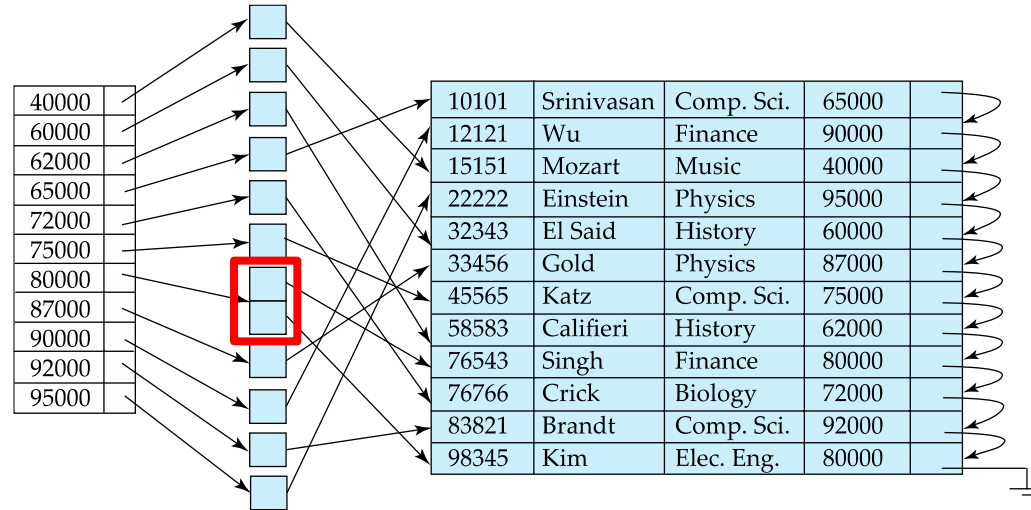


- For unclustered index: sparse index on top of dense index (multilevel index)



Secondary Indices Example

- Secondary index on salary field of instructor



- Index record points to a bucket that contains pointers to all the actual records with that particular search-key value.
- Secondary indices have to be dense



Clustering vs Nonclustering Indices

- Indices offer substantial benefits when searching for records.
- BUT: indices imposes overhead on database modification
 - when a record is inserted or deleted, every index on the relation must be updated
 - When a record is updated, any index on an updated attribute must be updated
- Sequential scan using clustering index is efficient, but a sequential scan using a secondary (nonclustering) index is expensive on magnetic disk
 - Each record access may fetch a new block from disk
 - Each block fetch on magnetic disk requires about 5 to 10 milliseconds



Indices on Multiple Keys

- **Composite search key**
 - E.g., index on *instructor* relation on attributes (*name*, *ID*)
 - Values are sorted lexicographically
 - E.g. (John, 12121) < (John, 13514) and (John, 13514) < (Peter, 11223)
 - Can query on just *name*, or on (*name*, *ID*)

- (nameLast, nameFirst, birthyear)
 - nameLast [nameLast = "Ferguson"] [nameLast like "Fer%"]
 - nameLast, nameFirst
 - nameLast, nameFirst, birthyear

- NOT and index on
 - nameFirst, nameLast
 - birthyear
 - nameLast like [%er%]

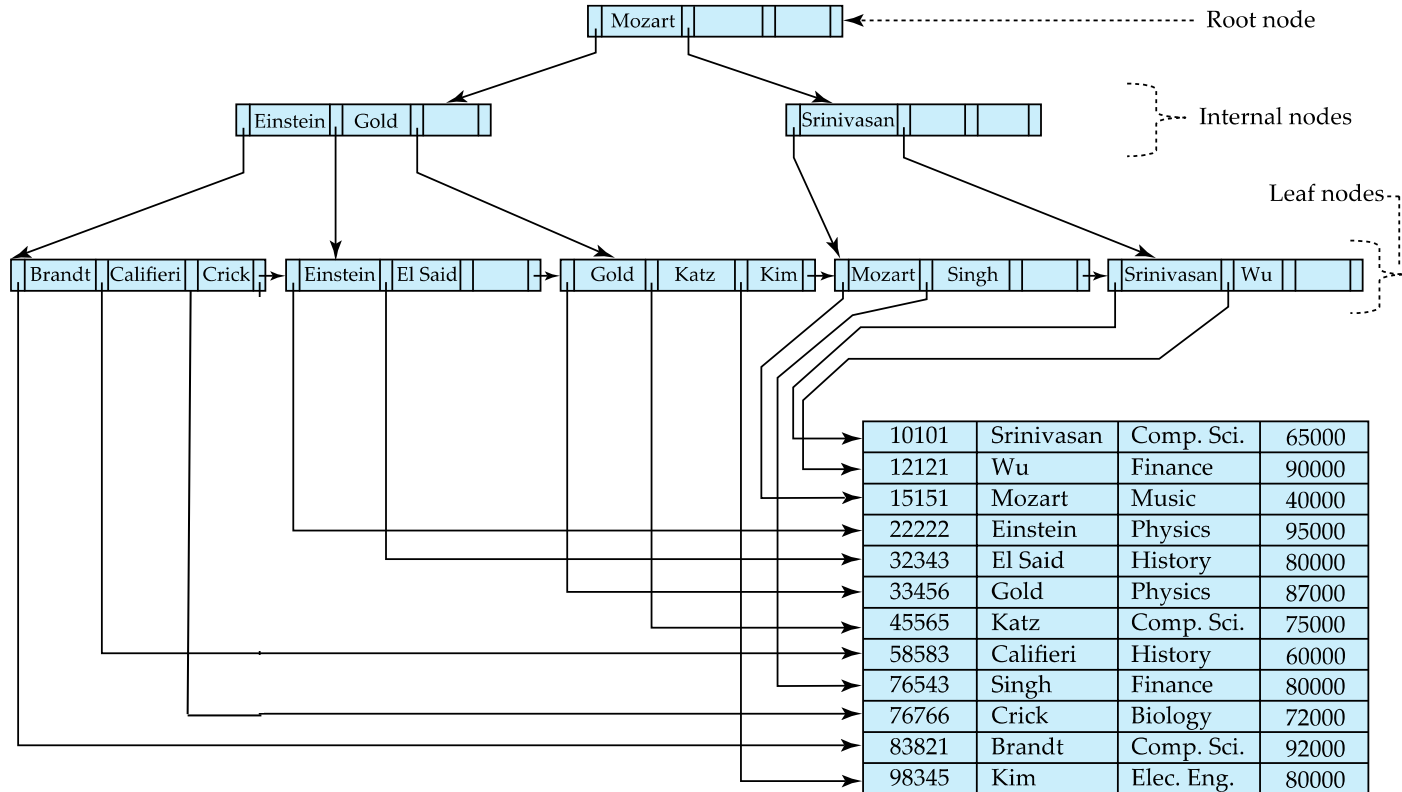


B⁺-Tree Index Files

- Disadvantage of indexed-sequential files
 - Performance degrades as file grows, since many overflow blocks get created.
 - Periodic reorganization of entire file is required.
- Advantage of B⁺-tree index files:
 - Automatically reorganizes itself with small, local, changes, in the face of insertions and deletions.
 - Reorganization of entire file is not required to maintain performance.
- (Minor) disadvantage of B⁺-trees:
 - Extra insertion and deletion overhead, space overhead.
- Advantages of B⁺-trees outweigh disadvantages
 - B⁺-trees are used extensively



Example of B⁺-Tree





B⁺-Tree Index Files (Cont.)

A B⁺-tree is a rooted tree satisfying the following properties:

- All paths from root to leaf are of the same length
- Each node that is not a root or a leaf has between $\lceil n/2 \rceil$ and n children.
- A leaf node has between $\lceil (n-1)/2 \rceil$ and $n-1$ values
- Special cases:
 - If the root is not a leaf, it has at least 2 children.
 - If the root is a leaf (that is, there are no other nodes in the tree), it can have between 0 and $(n-1)$ values.



B⁺-Tree Node Structure

- Typical node

P_1	K_1	P_2	$\dots$	P_{n-1}	K_{n-1}	P_n
-------	-------	-------	---------	-----------	-----------	-------

- K_i are the search-key values
 - P_i are pointers to children (for non-leaf nodes) or pointers to records or buckets of records (for leaf nodes).
- The search-keys in a node are ordered

$$K_1 < K_2 < K_3 < \dots < K_{n-1}$$

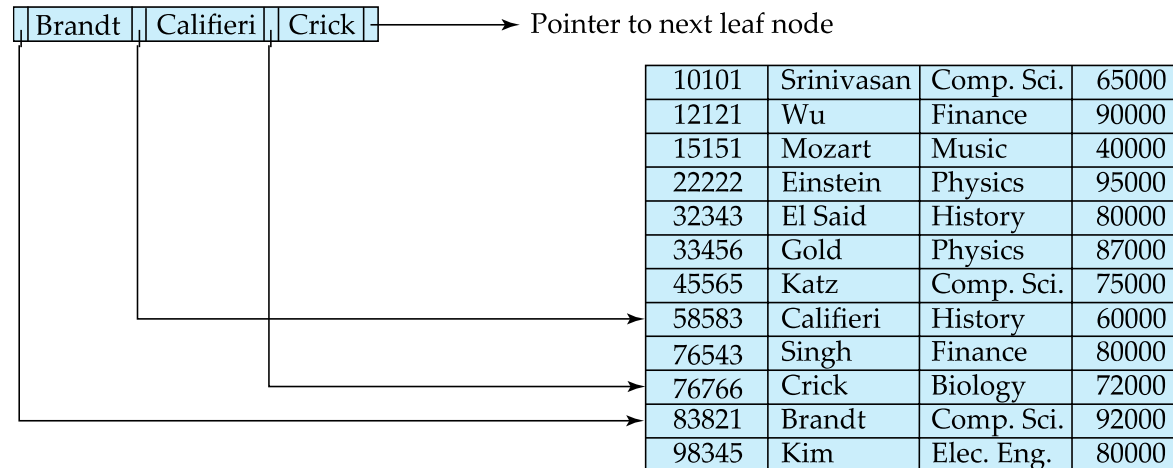
(Initially assume no duplicate keys, address duplicates later)



Leaf Nodes in B⁺-Trees

Properties of a leaf node:

- For $i = 1, 2, \dots, n-1$, pointer P_i points to a file record with search-key value K_i ,
- If L_i, L_j are leaf nodes and $i < j$, L_i 's search-key values are less than or equal to L_j 's search-key values
- P_n points to next leaf node in search-key order





Non-Leaf Nodes in B⁺-Trees

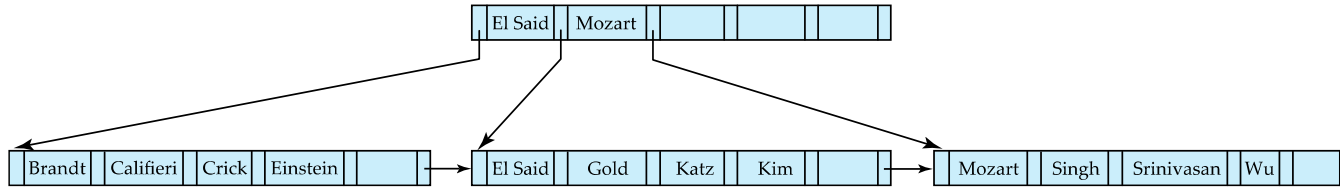
- Non leaf nodes form a multi-level sparse index on the leaf nodes. For a non-leaf node with m pointers:
 - All the search-keys in the subtree to which P_1 points are less than K_1
 - For $2 \leq i \leq n - 1$, all the search-keys in the subtree to which P_i points have values greater than or equal to K_{i-1} and less than K_i
 - All the search-keys in the subtree to which P_n points have values greater than or equal to K_{n-1}
 - General structure

P_1	K_1	P_2	$\dots$	P_{n-1}	K_{n-1}	P_n
-------	-------	-------	---------	-----------	-----------	-------



Example of B⁺-tree

- B⁺-tree for *instructor* file ($n = 6$)



- Leaf nodes must have between 3 and 5 values ($\lceil (n-1)/2 \rceil$ and $n-1$, with $n = 6$).
- Non-leaf nodes other than root must have between 3 and 6 children ($\lceil n/2 \rceil$ and n with $n = 6$).
- Root must have at least 2 children.



Observations about B⁺-trees

- Since the inter-node connections are done by pointers, “logically” close blocks need not be “physically” close.
- The non-leaf levels of the B⁺-tree form a hierarchy of sparse indices.
- The B⁺-tree contains a relatively small number of levels
 - Level below root has at least $2 * \lceil n/2 \rceil$ values
 - Next level has at least $2 * \lceil n/2 \rceil * \lceil n/2 \rceil$ values
 - .. etc.
 - If there are K search-key values in the file, the tree height is no more than $\lceil \log_{\lceil n/2 \rceil}(K) \rceil$
 - thus searches can be conducted efficiently.
- Insertions and deletions to the main file can be handled efficiently, as the index can be restructured in logarithmic time (as we shall see).

Show the Simulator

<https://www.cs.usfca.edu/~galles/visualization/BPlusTree.html>



Hashing



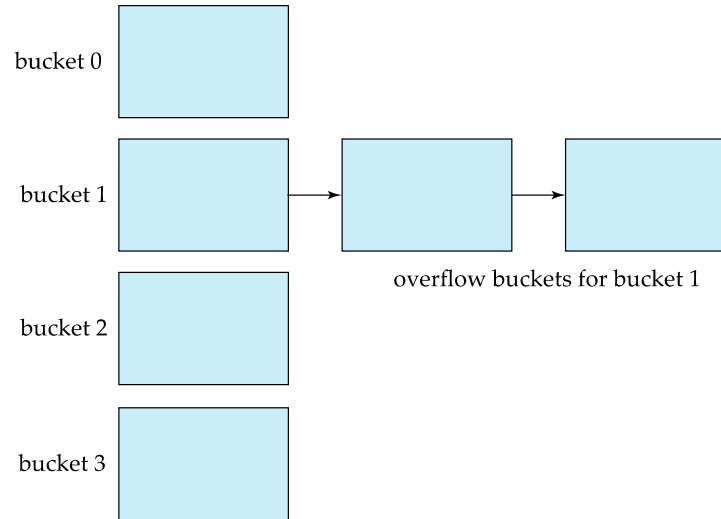
Static Hashing

- A **bucket** is a unit of storage containing one or more entries (a bucket is typically a disk block).
 - we obtain the bucket of an entry from its search-key value using a **hash function**
- Hash function h is a function from the set of all search-key values K to the set of all bucket addresses B .
- Hash function is used to locate entries for access, insertion as well as deletion.
- Entries with different search-key values may be mapped to the same bucket; thus entire bucket has to be searched sequentially to locate an entry.
- In a **hash index**, buckets store entries with pointers to records
- In a **hash file-organization** buckets store records



Handling of Bucket Overflows (Cont.)

- **Overflow chaining** – the overflow buckets of a given bucket are chained together in a linked list.
- Above scheme is called **closed addressing** (also called **closed hashing** or **open hashing** depending on the book you use)
 - An alternative, called **open addressing** (also called **open hashing** or **closed hashing** depending on the book you use) which does not use overflow buckets, is not suitable for database applications.





Example of Hash File Organization

Hash file organization of *instructor* file, using *dept_name* as key.

bucket 0

bucket 1

15151	Mozart	Music	40000

bucket 2

32343	El Said	History	80000
58583	Califieri	History	60000

bucket 3

22222	Einstein	Physics	95000
33456	Gold	Physics	87000
98345	Kim	Elec. Eng.	80000

bucket 4

12121	Wu	Finance	90000
76543	Singh	Finance	80000

bucket 5

76766	Crick	Biology	72000

bucket 6

10101	Srinivasan	Comp. Sci.	65000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000

bucket 7



Deficiencies of Static Hashing

- In static hashing, function h maps search-key values to a fixed set of B of bucket addresses. Databases grow or shrink with time.
 - If initial number of buckets is too small, and file grows, performance will degrade due to too much overflows.
 - If space is allocated for anticipated growth, a significant amount of space will be wasted initially (and buckets will be underfull).
 - If database shrinks, again space will be wasted.
- One solution: periodic re-organization of the file with a new hash function
 - Expensive, disrupts normal operations
- Better solution: allow the number of buckets to be modified dynamically.

Show the Simulator

<https://www.cs.usfca.edu/~galles/visualization/OpenHash.html>

<https://visualgo.net/en/hashtable>

<http://iswsa.acm.org/mphf/openDSAPerfectHashAnimation/perfectHashAV.html>

<https://opensa-server.cs.vt.edu/ODSA/AV/Development/hashAV.html>

Query Compilation

Query Compilation

Preview of Query Compilation

Database Systems: The Complete Book (2nd Edition) 2nd Edition

by [Hector Garcia-Molina](#) (Author), [Jeffrey D. Ullman](#) (Author), [Jennifer Widom](#) (Author)

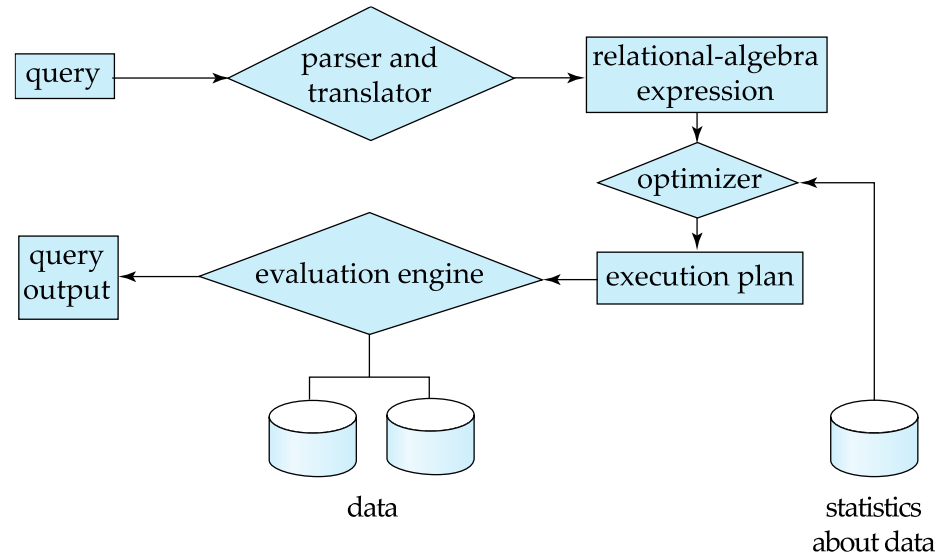
To set the context for query execution, we offer a very brief outline of the content of the next chapter. Query compilation is divided into the three major steps shown in Fig. 15.2.

- a) *Parsing*. A *parse tree* for the query is constructed.
- b) *Query Rewrite*. The parse tree is converted to an initial query plan, which is usually an algebraic representation of the query. This initial plan is then transformed into an equivalent plan that is expected to require less time to execute.
- c) *Physical Plan Generation*. The abstract query plan from (b), often called a *logical query plan*, is turned into a *physical query plan* by selecting algorithms to implement each of the operators of the logical plan, and by selecting an order of execution for these operators. The physical plan, like the result of parsing and the logical plan, is represented by an expression tree. The physical plan also includes details such as how the queried relations are accessed, and when and if a relation should be sorted.

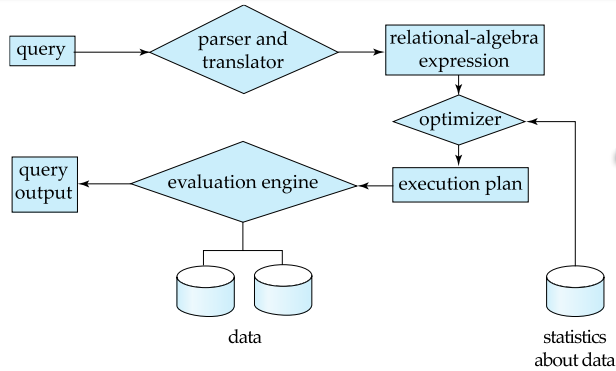


Basic Steps in Query Processing

1. Parsing and translation
2. Optimization
3. Evaluation

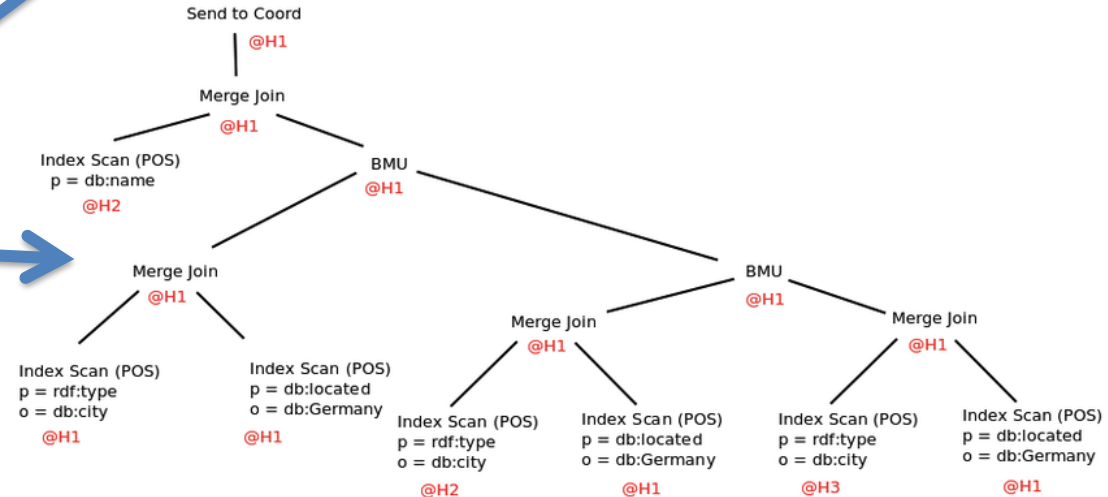
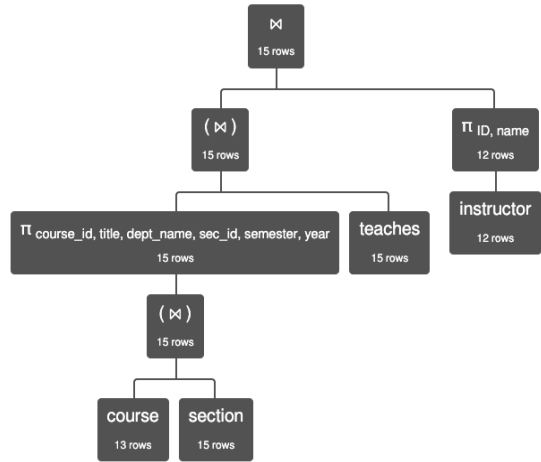


Query Processing

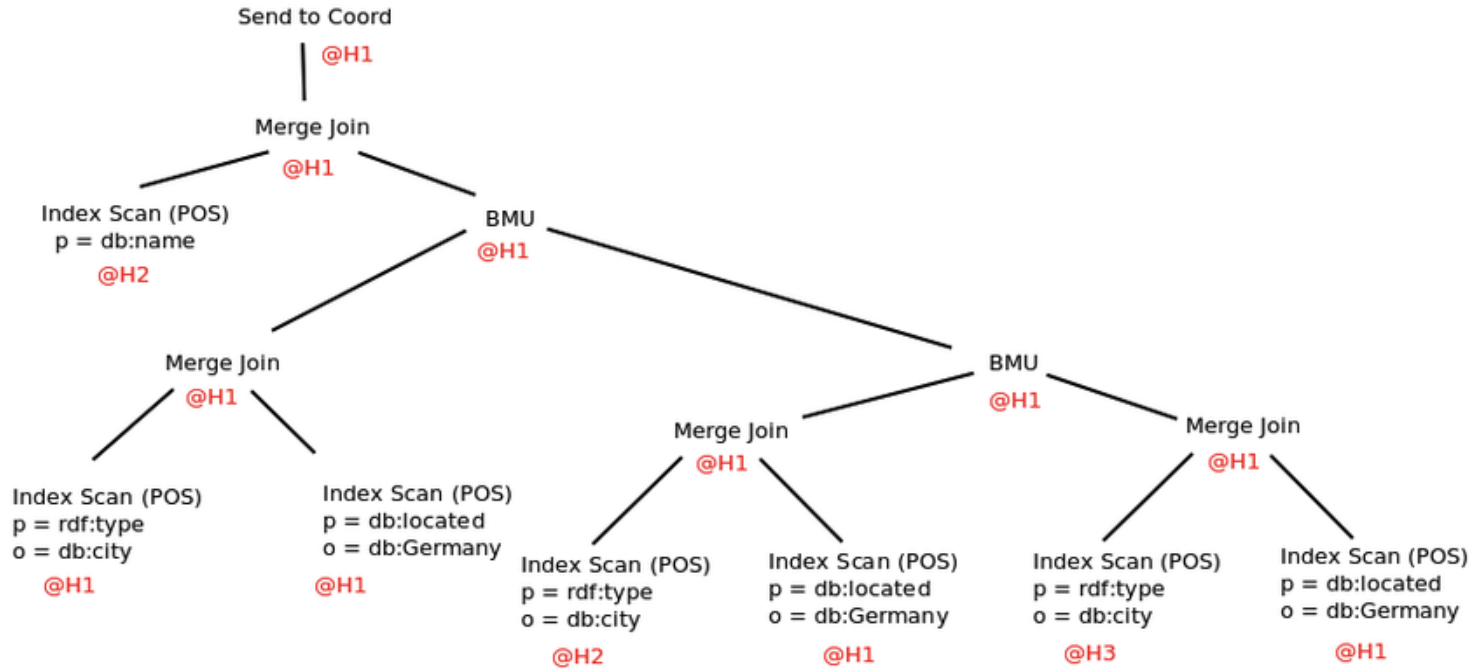


```

1 (
2   (π course_id, title, dept_name, sec_id, semester, year
3     (course ⋈ section)
4   )
5   ⋈ teaches
6 )
7 ⋈
8 (π ID, name (instructor))
  
```



Query Plan and Algorithms



Parsing and Execution

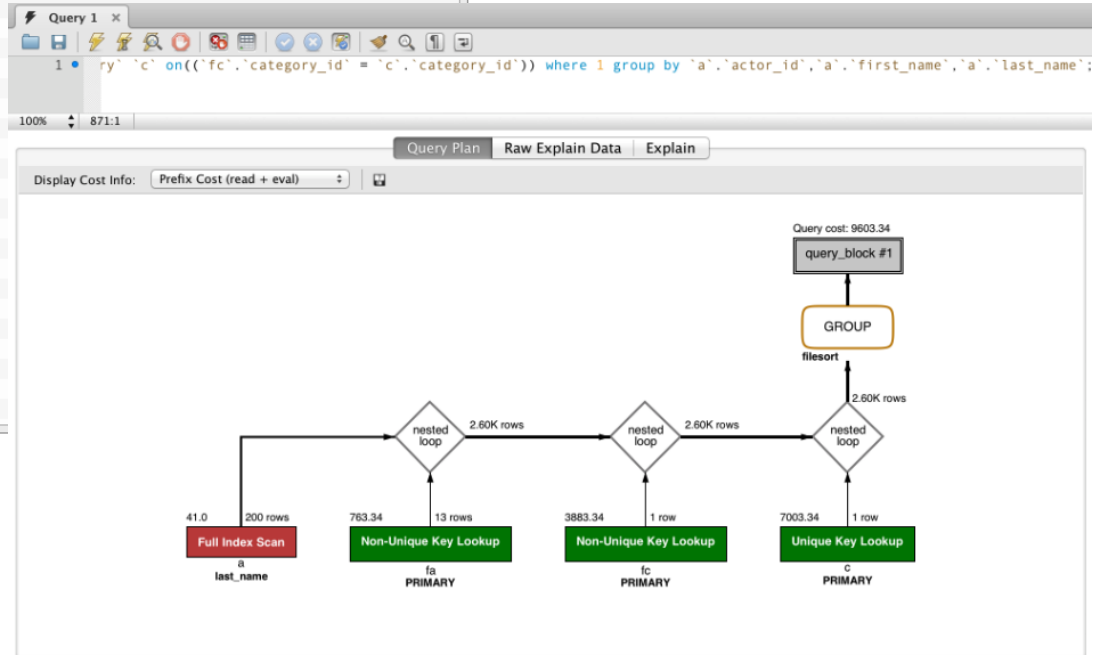
- Parser/Translator
 - Verifies syntax correctness and generates a *parse tree*.
 - Converts to *logical plan tree* that defines how to execute the query.
 - Tree nodes are *operator(tables, parameters)*
 - Edges are the flow of data “up the tree” from node to node.
- Optimizer
 - Modifies the logical plan to define an improved execution.
 - Query rewrite/transformation.
 - Determines *how* to choose among multiple implementations of operators.
- Engine
 - Executes the plan
 - May modify the plan to *optimize* execution, e.g. using indexes.

EXPLAIN Example

JSON EXPLAIN

Example JSON Formatted EXPLAIN

```
mysql> explain format=json select `a`.`actor_id` AS `actor_id`,`a`.`first_name` AS `first_name`,`a`.`last_name` AS `last_name`
***** 1. row *****
EXPLAIN: {
  "query_block": {
    "select_id": 1,
    "cost_info": {
      "query_cost": "9603.34"
    },
    "grouping_operation": {
      "using_filesort": true,
      "cost_info": {
        "sort_cost": "2600.00"
      },
      "nested_loop": [
        {
          "table": {
            "table_name": "a",
            "access_type": "index",
            "possible_keys": [
              "last_name",
              "ln_fn_idx"
            ],
            "key": "last_name",
            "used_key_parts": [
              "last_name",
              "first_name"
            ]
          }
        }
      ]
    }
  }
}
```





Basic Steps in Query Processing: Optimization

- A relational algebra expression may have many equivalent expressions
 - E.g., $\sigma_{salary < 75000}(\Pi_{salary}(instructor))$ is equivalent to $\Pi_{salary}(\sigma_{salary < 75000}(instructor))$
- Each relational algebra operation can be evaluated using one of several different algorithms
 - Correspondingly, a relational-algebra expression can be evaluated in many ways.
- Annotated expression specifying detailed evaluation strategy is called an **evaluation-plan**. E.g.,:
 - Use an index on *salary* to find instructors with salary < 75000,
 - Or perform complete relation scan and discard instructors with salary ≥ 75000



Basic Steps: Optimization (Cont.)

- **Query Optimization:** Amongst all equivalent evaluation plans choose the one with lowest cost.
 - Cost is estimated using statistical information from the database catalog
 - e.g.. number of tuples in each relation, size of tuples, etc.
- In this chapter we study
 - How to measure query costs
 - Algorithms for evaluating relational algebra operations
 - How to combine algorithms for individual operations in order to evaluate a complete expression
- In Chapter 16
 - We study how to optimize queries, that is, how to find an evaluation plan with lowest estimated cost

Query Cost



Measures of Query Cost

- Many factors contribute to time cost
 - *disk access, CPU, and network communication*
- Cost can be measured based on
 - **response time**, i.e. total elapsed time for answering query, or
 - total **resource consumption**
- We use total resource consumption as cost metric
 - Response time harder to estimate, and minimizing resource consumption is a good idea in a shared database
- We ignore CPU costs for simplicity
 - Real systems do take CPU cost into account
 - Network costs must be considered for parallel systems
- We describe how estimate the cost of each operation
 - We do not include cost to writing output to disk



Measures of Query Cost

If I ask a query cost estimation question on a HW or exam, you only need to estimate the number of block transfers.

- Disk cost can be estimated as:
 - Number of seeks * average-seek-cost
 - Number of blocks read * average-block-read-cost
 - Number of blocks written * average-block-write-cost
- For simplicity we just use the **number of block transfers** from disk and the **number of seeks** as the cost measures
 - t_T – time to transfer one block
 - Assuming for simplicity that write cost is same as read cost
 - t_S – time for one seek
 - Cost for b block transfers plus S seeks
$$b * t_T + S * t_S$$
- t_S and t_T depend on where data is stored; with 4 KB blocks:
 - High end magnetic disk: $t_S = 4$ msec and $t_T = 0.1$ msec
 - SSD: $t_S = 20$ -90 microsec and $t_T = 2$ -10 microsec for 4KB



Measures of Query Cost (Cont.)

- Required data may be buffer resident already, avoiding disk I/O
 - But hard to take into account for cost estimation
- Several algorithms can reduce disk IO by using extra buffer space
 - Amount of real memory available to buffer depends on other concurrent queries and OS processes, known only during execution
- Worst case estimates assume that no data is initially in buffer and only the minimum amount of memory needed for the operation is available
 - But more optimistic estimates are used in practice

Algorithm Selection for SELECT



Selection Operation

This seems to assume that all blocks for the relation are on the same cylinder.

- **File scan**
- Algorithm **A1 (linear search)**. Scan each file block and test all records to see whether they satisfy the selection condition.
 - Cost estimate = b_r block transfers + 1 seek
 - b_r denotes number of blocks containing records from relation r
 - If selection is on a key attribute, can stop on finding record
 - cost = $(b_r/2)$ block transfers + 1 seek
 - Linear search can be applied regardless of
 - selection condition or
 - ordering of records in the file, or
 - availability of indices
- Note: binary search generally does not make sense since data is not stored consecutively
 - except when there is an index available,
 - and binary search requires more seeks than index search



Selections Using Indices

- **Index scan** – search algorithms that use an index
 - selection condition must be on search-key of index.
- **A2 (clustering index, equality on key)**. Retrieve a single record that satisfies the corresponding equality condition
 - $Cost = (h_i + 1) * (t_T + t_S)$
- **A3 (clustering index, equality on nonkey)** Retrieve multiple records.
 - Records will be on consecutive blocks
 - Let b = number of blocks containing matching records
 - $Cost = h_i * (t_T + t_S) + t_S + t_T * b$



Selections Using Indices

- **Index scan** – search algorithms that use an index
 - selection condition must be on search-key of index.
- **A2 (clustering index, equality on key)**. Retrieve a single record that satisfies the corresponding equality condition

- $Cost = (h_i + 1) * (t_T + t_S)$

- **A3 (clustering index, equality on nonkey)** Retrieve multiple records.

- Records will be on consecutive blocks
 - Let b = number of blocks containing matching records

- $Cost = h_i * (t_T + t_S) + t_S + t_T * b$

- Replace $(t_T + t_S)$ with t_B , the time/cost for reading a block.
- In my way of thinking, h_i is the “height” of the index tree.
- The cost to find the index block is $h_i * t_B$. We then read the referenced data block, which happens once because it is a key.
- If it is a non-key, we may retrieve multiple records → reading multiple blocks.
- But, clustering index on the non-key field → the data records are sorted by the non-key → records are probably on consecutive sectors on the same track → one seek and then some number of block reads.





Selections Using Indices

- **A4 (secondary index, equality on key/non-key).**
 - Retrieve a single record if the search-key is a candidate key
 - $Cost = (h_i + 1) * (t_T + t_S)$
 - Retrieve multiple records if search-key is not a candidate key
 - each of n matching records may be on a different block
 - $Cost = (h_i + n) * (t_T + t_S)$
 - Can be very expensive!

- $h_i * t_B$ to get the leaf node in the index.
- Each entry in the leaf node may reference a different block in the data file.
- If there are n matching records, we get
- $(h_i + n) * t_B$



Selections Involving Comparisons

- Can implement selections of the form $\sigma_{A \leq V}(r)$ or $\sigma_{A \geq V}(r)$ by using
 - a linear file scan,
 - or by using indices in the following ways:
- **A5 (clustering index, comparison).** (Relation is sorted on A)
 - For $\sigma_{A \geq V}(r)$ use index to find first tuple $\geq v$ and scan relation sequentially from there
 - For $\sigma_{A \leq V}(r)$ just scan relation sequentially till first tuple $> v$; do not use index
- **A6 (clustering index, comparison).**
 - For $\sigma_{A \geq V}(r)$ use index to find first index entry $\geq v$ and scan index sequentially from there, to find pointers to records.
 - For $\sigma_{A \leq V}(r)$ just scan leaf pages of index finding pointers to records, till first entry $> v$
 - In either case, retrieve records that are pointed to
 - requires an I/O per record; Linear file scan may be cheaper!



Implementation of Complex Selections

- **Conjunction:** $\sigma_{\theta_1 \wedge \theta_2 \wedge \dots \wedge \theta_n}(r)$
- **A7 (conjunctive selection using one index).**
 - Select a combination of θ_i and algorithms A1 through A7 that results in the least cost for $\sigma_{\theta_i}(r)$.
 - Test other conditions on tuple after fetching it into memory buffer.
- **A8 (conjunctive selection using composite index).**
 - Use appropriate composite (multiple-key) index if available.
- **A9 (conjunctive selection by intersection of identifiers).**
 - Requires indices with record pointers.
 - Use corresponding index for each condition, and take intersection of all the obtained sets of record pointers.
 - Then fetch records from file
 - If some conditions do not have appropriate indices, apply test in memory.



Algorithms for Complex Selections

- **Disjunction:** $\sigma_{\theta_1 \vee \theta_2 \vee \dots \vee \theta_n}(r)$.
- **A10 (disjunctive selection by union of identifiers).**
 - Applicable if *all* conditions have available indices.
 - Otherwise use linear scan.
 - Use corresponding index for each condition, and take union of all the obtained sets of record pointers.
 - Then fetch records from file
- **Negation:** $\sigma_{\neg\theta}(r)$
 - Use linear scan on file
 - If very few records satisfy $\neg\theta$, and an index is applicable to θ
 - Find satisfying records using index and fetch from file

Conjunctive and Disjunctive



- Assume there is an index on *name* but not on *dept_name*.
- Consider two simple use cases
 - *select * from student where name="Smith" AND dept_name="Comp. Sci."*
 - *select * from student where name="Smith" OR dept_name="Comp. Sci."*
- With a conjunction, the engine must
 - Find and read the blocks with the correct name.
 - Scan each block to evaluate the rest of the AND, but this is not an I/O.
- The index does not help for the disjunction because the engine still needs to scan the data file and read blocks looking for the matching OR conditions.

Algorithm Selection for JOIN



Join Operation

- Several different algorithms to implement joins
 - Nested-loop join
 - Block nested-loop join
 - Indexed nested-loop join
 - Merge-join
 - Hash-join
- Choice based on cost estimate
- Examples use the following information
 - Number of records of *student*: 5,000 *takes*: 10,000
 - Number of blocks of *student*: 100 *takes*: 400



Nested-Loop Join

- To compute the theta join $r \bowtie_{\theta} s$
 for each tuple t_r **in** r **do begin**
 for each tuple t_s **in** s **do begin**
 test pair (t_r, t_s) to see if they satisfy the join condition θ
 if they do, add $t_r \cdot t_s$ to the result.
 end
 end
- r is called the **outer relation** and s the **inner relation** of the join.
- Requires no indices and can be used with any kind of join condition.
- Expensive since it examines every pair of tuples in the two relations.

For clarity (or confusion), I sometimes use the terms:

- Scan table for the outer relation
- Probe table for the inner relation



Nested-Loop Join (Cont.)

- In the worst case, if there is enough memory only to hold one block of each relation, the estimated cost is

$n_r * b_s + b_r$ block transfers, plus $n_r + b_r$ seeks

- If the smaller relation fits entirely in memory, use that as the inner relation.

- Reduces cost to $b_r + b_s$ block transfers and 2 seeks

- Assuming worst case memory availability cost estimate is

- with *student* as outer relation:

- $5000 * 400 + 100 = 2,000,100$ block transfers,

- $5000 + 100 = 5100$ seeks

- with *takes* as the outer relation

- $10000 * 100 + 400 = 1,000,400$ block transfers and 10,400 seeks

- If smaller relation (*student*) fits entirely in memory, the cost estimate will be 500 block transfers.
- Block nested-loops algorithm (next slide) is preferable.

R JOIN S on (...)

S JOIN R on (...)

Well, That was a Clear as Mud

- In the worst case, if there is enough memory only to hold one block of each relation, the estimated cost is

$n_r * b_s + b_r$ block transfers, plus $n_r + b_r$ seeks

- SELECT * from L JOIN R on L.x=R.y
- The algorithm reads each block of L one time and never reads it again in JOIN.
- For each record l in L, the algorithm must read each block of R one time.
- The number of blocks in a relation S is, over simplistically,
 - #(*S*) is the number of records.
 - *s*(*S*) is the average record size.
 - *S*(*b*) is the block size ➔ number of blocks is #(*S*)/(*s*(*b*)/*s*(*S*)), which is $O(\#(S))$
 - So, the number of blocks read is $O(\#(L)) * O(\#(R)) = O(\#(L) * \#(R))$
- “shmath” – Something that is completely made up but presented as precise and accurate using math symbols.





Indexed Nested-Loop Join

- Index lookups can replace file scans if
 - join is an equi-join or natural join and
 - an index is available on the inner relation's join attribute
 - Can construct an index just to compute a join.
- For each tuple t_r in the outer relation r , use the index to look up tuples in s that satisfy the join condition with tuple t_r .
- Worst case: buffer has space for only one page of r , and, for each tuple in r , we perform an index lookup on s .
- Cost of the join: $b_r (t_r + t_s) + n_r * c$
 - Where c is the cost of traversing index and fetching all matching s tuples for one tuple of r
 - c can be estimated as cost of a single selection on s using the join condition.
- If indices are available on join attributes of both r and s , use the relation with fewer tuples as the outer relation.

$$r.X = s.Y \text{ AND } r.P = s.Q$$
$$(X, Y) \rightarrow (P, Q)$$



Sorting

- We may build an index on the relation, and then use the index to read the relation in sorted order. May lead to one disk block access for each tuple.
 - For relations that fit in memory, techniques like quicksort can be used.
 - For relations that don't fit in memory, **external sort-merge** is a good choice.
-
- The engine may build either a sorted index or a hash index.
 - The key idea is:
 - The cost of building the index and then using the index in the algorithm
 - Is cheaper than running the default algorithm with an index

We Really Hate this Class

- One way to *overly* simplify these analyses is realizing that the no. of blocks is “linear” with the number of records/tuples in a relation. →
 - Overly simplify and assume that the block size is one record.
 - Analyze using the number of reads as a function of no. of rows.
- Assume that relation R has $\#(R)$ rows:
 - A tree-based lookup when you can use a key is $O(\log(\#R))$.
The base of the logarithm does not materially change the analysis.
 - A hash-based lookup is $O(1)$.
 - A scan or scan-based look up is $O(\#R)$.
- Index creation cost:
 - Hash is $O(\#(R))$
 - Tree is $O(\#(R) * \log(\#R))$
- So, very roughly an index join is either
 - $O(\#(L) * \log(\#(R)) + O(\#(R) * \log(\#(R))) = \log(\#(R)) * (\#(L) + \#(R))$ or
 - $O(\#(L) + \#(R))$, but this only works for equality.



$\#(R) = 100$

$\#(S) = 1000000$

Build index = 6 million

700 + 6 million



Merge-Join

1. Sort both relations on their join attribute (if not already sorted on the join attributes).
2. Merge the sorted relations to join them
 1. Join step is similar to the merge stage of the sort-merge algorithm.
 2. Main difference is handling of duplicate values in join attribute — every pair with same value on join attribute must be matched
 3. Detailed algorithm in book

	$a1$	$a2$		$a1$	$a3$																								
pr	<table><tr><td>a</td><td>3</td></tr><tr><td>b</td><td>1</td></tr><tr><td>d</td><td>8</td></tr><tr><td>d</td><td>13</td></tr><tr><td>f</td><td>7</td></tr><tr><td>m</td><td>5</td></tr><tr><td>q</td><td>6</td></tr></table>		a	3	b	1	d	8	d	13	f	7	m	5	q	6	ps	<table><tr><td>a</td><td>A</td></tr><tr><td>b</td><td>G</td></tr><tr><td>c</td><td>L</td></tr><tr><td>d</td><td>N</td></tr><tr><td>m</td><td>B</td></tr></table>		a	A	b	G	c	L	d	N	m	B
a	3																												
b	1																												
d	8																												
d	13																												
f	7																												
m	5																												
q	6																												
a	A																												
b	G																												
c	L																												
d	N																												
m	B																												
				s																									
	r																												

$\#(A) = n, \#(B) = m$

$n * m$

$n * \log(n) + m * \log(m) + n + m$



Merge-Join (Cont.)

- Can be used only for equi-joins and natural joins
- Each block needs to be read only once (assuming all tuples for any given value of the join attributes fit in memory)
- Thus the cost of merge join is:
$$b_r + b_s \text{ block transfers} + \lceil b_r / b_b \rceil + \lceil b_s / b_b \rceil \text{ seeks}$$

+ the cost of sorting if relations are unsorted.
- **hybrid merge-join:** If one relation is sorted, and the other has a secondary B⁺-tree index on the join attribute
 - Merge the sorted relation with the leaf entries of the B⁺-tree .
 - Sort the result on the addresses of the unsorted relation's tuples
 - Scan the unsorted relation in physical address order and merge with previous result, to replace addresses by the actual tuples
 - Sequential scan more efficient than random lookup



Hash-Join

- Applicable for equi-joins and natural joins.
- A hash function h is used to partition tuples of both relations
- h maps *JoinAttrs* values to $\{0, 1, \dots, n\}$, where *JoinAttrs* denotes the common attributes of r and s used in the natural join.
 - $r_0, r_1, \dots, r_n$ denote partitions of r tuples
 - Each tuple $t_r \in r$ is put in partition r_i where $i = h(t_r[\text{JoinAttrs}])$.
 - $r_0, r_1, \dots, r_n$ denotes partitions of s tuples
 - Each tuple $t_s \in s$ is put in partition s_i , where $i = h(t_s[\text{JoinAttrs}])$.
- *Note:* In book, Figure 12.10 r_i is denoted as H_{r_i} , s_i is denoted as H_{s_i} and n is denoted as n_h .

On $A.x=b.y$ and $A.z=b.q$

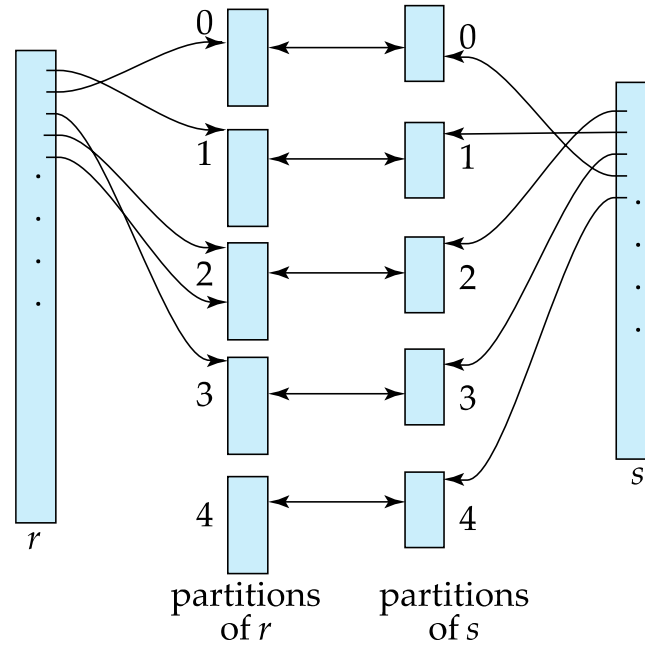
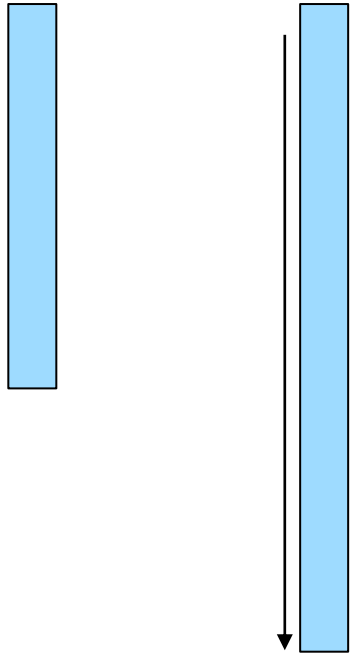
(z,q)

(x,y)



Hash-Join (Cont.)

On $a.X=b.Z$





Hash-Join Algorithm

The hash-join of r and s is computed as follows.

1. Partition the relation s using hashing function h . When partitioning a relation, one block of memory is reserved as the output buffer for each partition.
2. Partition r similarly.
3. For each i :
 - (a) Load s_i into memory and build an in-memory hash index on it using the join attribute. This hash index uses a different hash function than the earlier one h .
 - (b) Read the tuples in r_i from the disk one by one. For each tuple t_r locate each matching tuple t_s in s_i using the in-memory hash index. Output the concatenation of their attributes.

Relation s is called the **build input** and r is called the **probe input**.



Hash-Join algorithm (Cont.)

- The value n and the hash function h is chosen such that each s_i should fit in memory.
 - Typically n is chosen as $\lceil b_s/M \rceil * f$ where f is a “**fudge factor**”, typically around 1.2
 - The probe relation partitions s_i need not fit in memory
- **Recursive partitioning** required if number of partitions n is greater than number of pages M of memory.
 - instead of partitioning n ways, use $M - 1$ partitions for s
 - Further partition the $M - 1$ partitions using a different hash function
 - Use same partitioning method on r
 - Rarely required: e.g., with block size of 4 KB, recursive partitioning not needed for relations of < 1 GB with memory size of 2MB, or relations of < 36 GB with memory of 12 MB

Algorithm Selection for Other Operations



Other Operations

- **Duplicate elimination** can be implemented via hashing or sorting.
 - On sorting duplicates will come adjacent to each other, and all but one set of duplicates can be deleted.
 - *Optimization*: duplicates can be deleted during run generation as well as at intermediate merge steps in external sort-merge.
 - Hashing is similar – duplicates will come into the same bucket.
- **Projection**:
 - perform projection on each tuple
 - followed by duplicate elimination.

Select b,c,f from A
Union
Select a,q,r from B



Other Operations : Aggregation

- **Aggregation** can be implemented in a manner similar to duplicate elimination.
 - **Sorting** or **hashing** can be used to bring tuples in the same group together, and then the aggregate functions can be applied on each group.
 - Optimization: **partial aggregation**
 - combine tuples in the same group during run generation and intermediate merges, by computing partial aggregate values
 - For count, min, max, sum: keep aggregate values on tuples found so far in the group.
 - When combining partial aggregate for count, add up the partial aggregates
 - For avg, keep sum and count, and divide sum by count at the end

Evaluation



Evaluation of Expressions

- So far: we have seen algorithms for individual operations
- Alternatives for evaluating an entire expression tree
 - **Materialization**: generate results of an expression whose inputs are relations or are already computed, **materialize** (store) it on disk. Repeat.
 - **Pipelining**: pass on tuples to parent operations even as an operation is being executed
- We study above alternatives in more detail

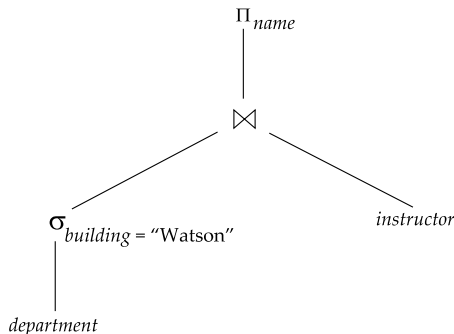


Materialization

- **Materialized evaluation:** evaluate one operation at a time, starting at the lowest-level. Use intermediate results materialized into temporary relations to evaluate next-level operations.
- E.g., in figure below, compute and store

$$\sigma_{\text{building}=\text{"Watson"}}(\textit{department})$$

then compute the store its join with *instructor*, and finally compute the projection on *name*.





Materialization (Cont.)

- Materialized evaluation is always applicable
- Cost of writing results to disk and reading them back can be quite high
 - Our cost formulas for operations ignore cost of writing results to disk, so
 - Overall cost = Sum of costs of individual operations + cost of writing intermediate results to disk
- **Double buffering:** use two output buffers for each operation, when one is full write it to disk while the other is getting filled
 - Allows overlap of disk writes with computation and reduces execution time



Pipelining

- **Pipelined evaluation:** evaluate several operations simultaneously, passing the results of one operation on to the next.
- E.g., in previous expression tree, don't store result of
$$\sigma_{building="Watson"}(department)$$
 - instead, pass tuples directly to the join.. Similarly, don't store result of join, pass tuples directly to projection.
- Much cheaper than materialization: no need to store a temporary relation to disk.
- Pipelining may not always be possible – e.g., sort, hash-join.
- For pipelining to be effective, use evaluation algorithms that generate output tuples even as tuples are received for inputs to the operation.
- Pipelines can be executed in two ways: **demand driven** and **producer driven**



Pipelining (Cont.)

- In **demand driven** or **lazy** evaluation
 - system repeatedly requests next tuple from top level operation
 - Each operation requests next tuple from children operations as required, in order to output its next tuple
 - In between calls, operation has to maintain “**state**” so it knows what to return next
- In **producer-driven** or **eager** pipelining
 - Operators produce tuples eagerly and pass them up to their parents
 - Buffer maintained between operators, child puts tuples in buffer, parent removes tuples from buffer
 - if buffer is full, child waits till there is space in the buffer, and then generates more tuples
 - System schedules operations that have space in output buffer and can process more input tuples
- Alternative name: **pull** and **push** models of pipelining



Pipelining (Cont.)

- Implementation of demand-driven pipelining
 - Each operation is implemented as an **iterator** implementing the following operations
 - **open()**
 - E.g., file scan: initialize file scan
 - state: pointer to beginning of file
 - E.g., merge join: sort relations;
 - state: pointers to beginning of sorted relations
 - **next()**
 - E.g., for file scan: Output next tuple, and advance and store file pointer
 - E.g., for merge join: continue with merge from earlier state till next output tuple is found. Save pointers as iterator state.
 - **close()**

NoSQL

Reminder

Simplistic Classification (<https://medium.com/swlh/4-types-of-nosql-databases-d88ad21f7d3b>)

Relational is the foundational model.

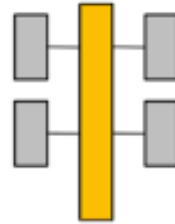
We covered graphs and examples.

SQL Database

Relational



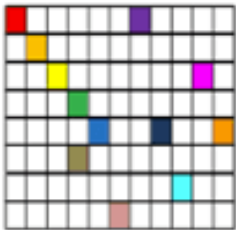
Analytical (OLAP)



We will see OLAP in a future lecture.

NoSQL Database

Column-Family



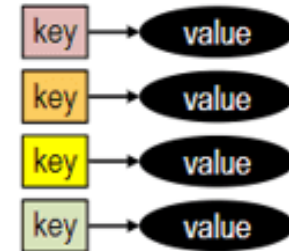
Graph



Document



Key-Value

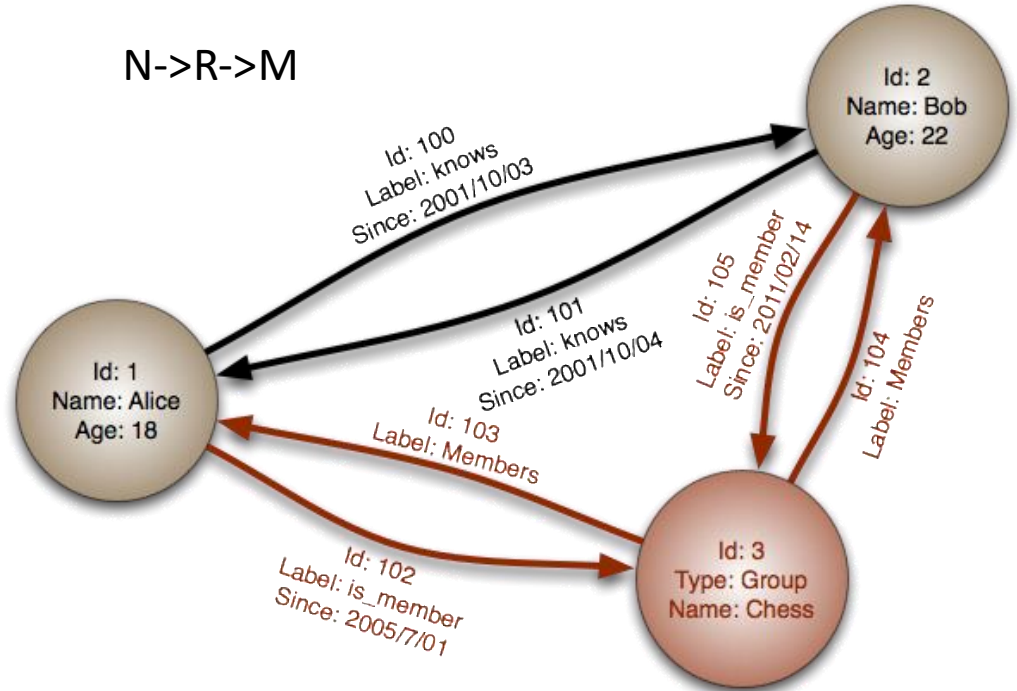


Subject of this lecture and part of HW3

Graphs

Graph Database

- Exactly what it sounds like
- Two core types
 - Node
 - Edge (link)
- Nodes and Edges have
 - Label(s) = “Kind”
 - Properties (free form)
- Query is of the form
 - $p1(n)-p2(e)-p3(m)$
 - n, m are nodes; e is an edge
 - $p1, p2, p3$ are predicates on labels and properties

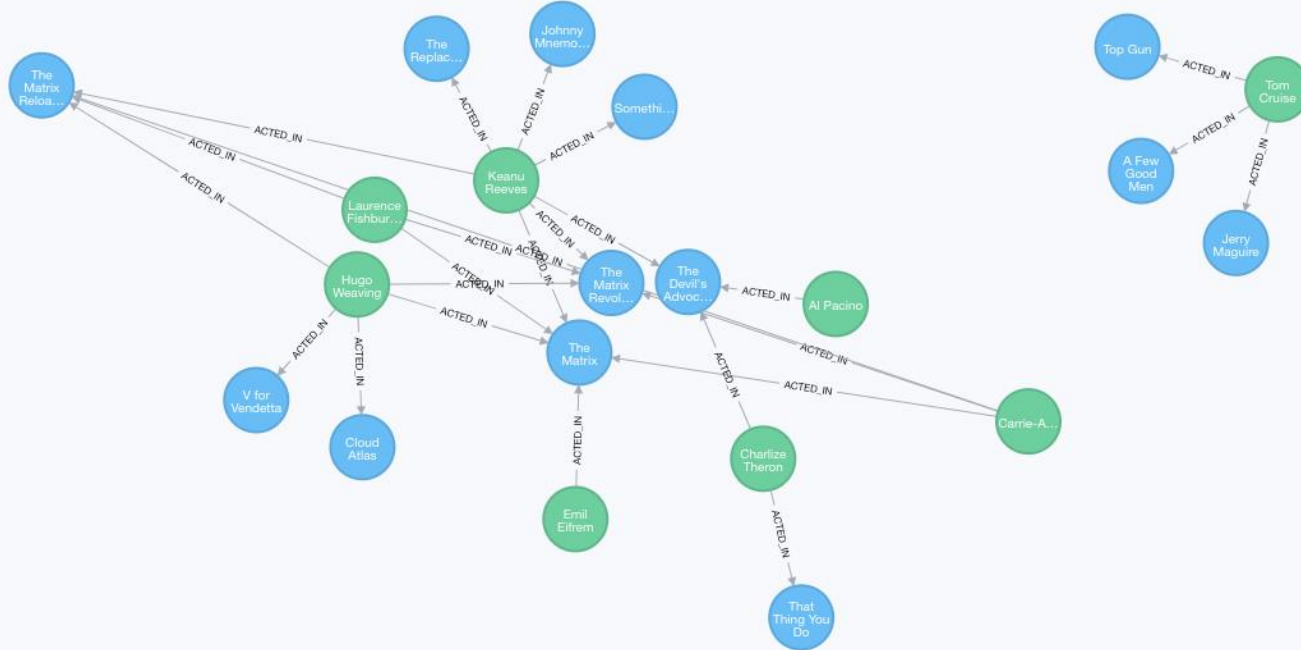


Neo4J Graph Query

```
$ MATCH p=()-[r:ACTED_IN]->() RETURN p LIMIT 25
```

*(21) Movie(13) Person(8)

*(25) ACTED_IN(25)



Displaying 21 nodes, 25 relationships.

AUTO-COMPLETE ☒

Why Graph Databases?

- Schema Less and Efficient storage of Semi Structured Information
- No O/R mismatch – very natural to map a graph to an Object Oriented language like Ruby.
- Express Queries as Traversals. Fast deep traversal instead of slow SQL queries that span many table joins.
- Very natural to express graph related problem with traversals (recommendation engine, find shortest path etc..)
- Seamless integration with various existing programming languages.
- ACID Transaction with rollbacks support.
- Whiteboard friendly – you use the language of node, properties and relationship to describe your domain (instead of e.g. UML) and there is no need to have a complicated O/R mapping tool to implement it in your database. You can say that Neo4j is “Whiteboard friendly” !(<http://video.neo4j.org/JHU6F/live-graph-session-how-allison-knows-james/>)

Graph Databases

Graph databases are

- Extremely fast for some queries and data models.
- Implement a language that vastly simplifies writing queries.

Social Network “path exists” Performance

- Experiment:
 - ~1k persons
 - Average 50 friends per person
 - `pathExists(a,b)`
limited to depth 4

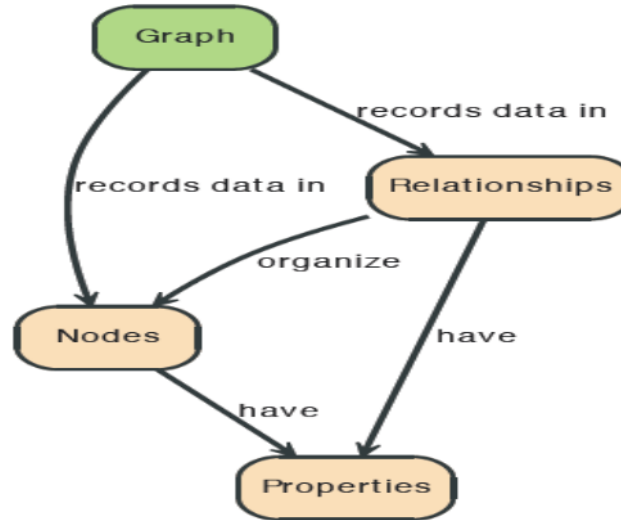
	# persons	query time
Relational database	1000	2000ms
Neo4j	1000	2ms
Neo4j	1000000	2ms

What are graphs good for?

- Recommendations
- Business intelligence
- Social computing
- Geospatial
- Systems management
- Web of things
- Genealogy
- Time series data
- Product catalogue
- Web analytics
- Scientific computing (especially bioinformatics)
- Indexing your *slow* RDBMS
- And much more!

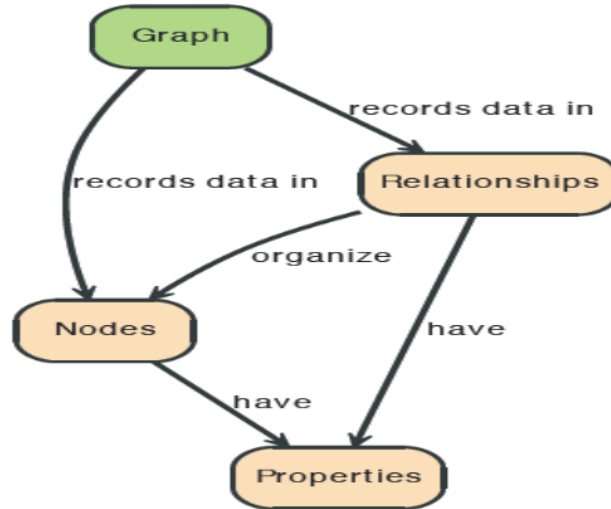
Graphs

- “A Graph —records data in→ Nodes —which have→ Properties”



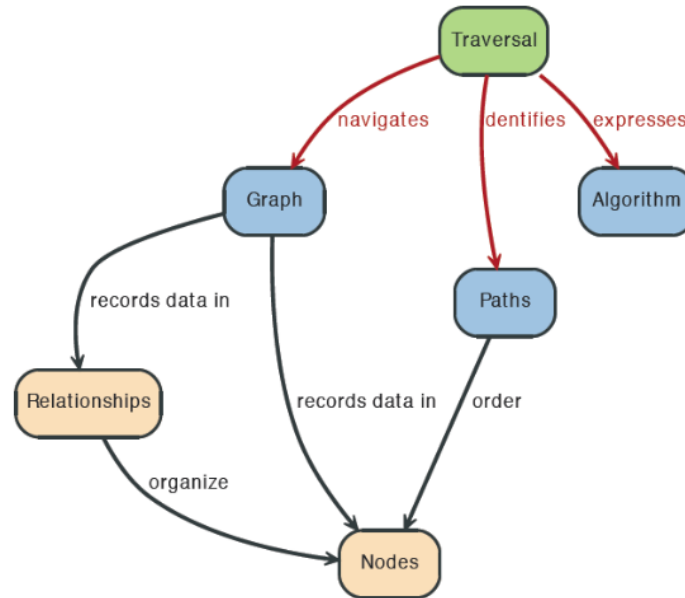
Graphs

- “Nodes —are organized by→ Relationships — which also have→ Properties”



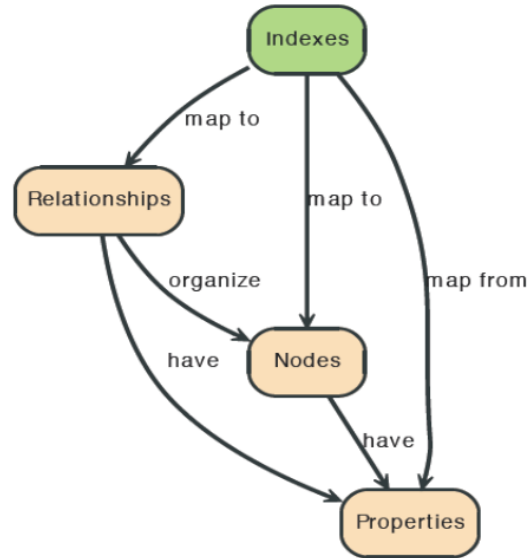
Query a graph with Traversal

- “A Traversal —navigates→ a Graph; it —identifies→ Paths —which order→ Nodes”



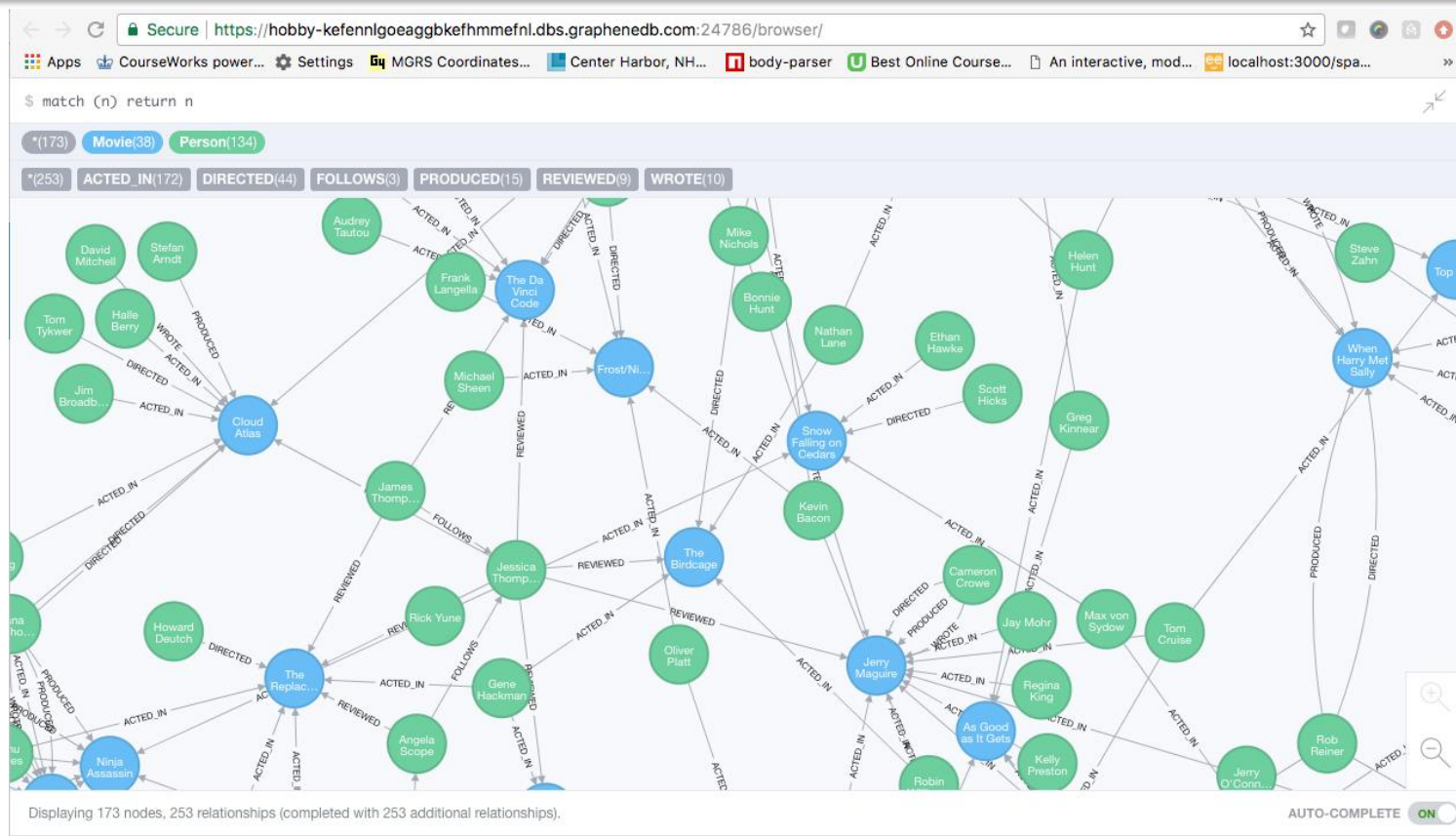
Indexes

- “An Index —maps from→ Properties —to either→ Nodes or Relationships”



Neo4j

A Graph Database (Sample)



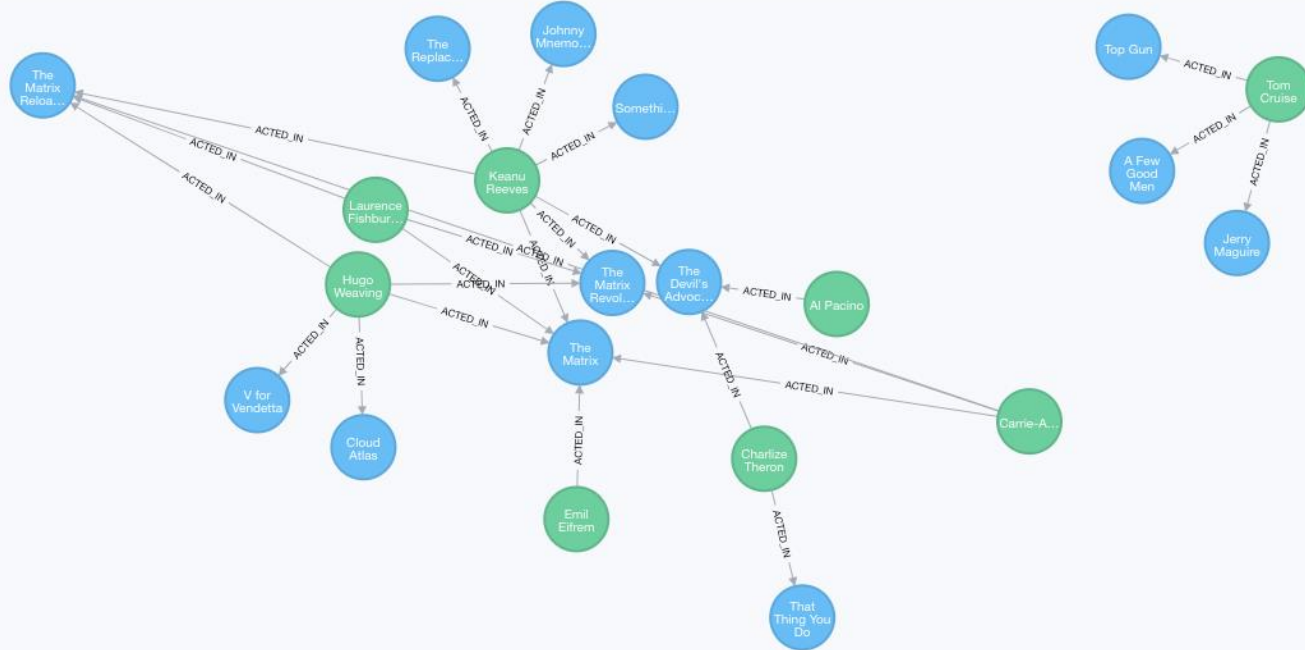
Neo4J Graph Query

Who acted in which movies?

```
$ MATCH p()-[r:ACTED_IN]->() RETURN p LIMIT 25
```

*(21) Movie(13) Person(8)

*(25) ACTED_IN(25)



Displaying 21 nodes, 25 relationships.

AUTO-COMPLETE ☒

Big Deal. That is just a JOIN.

- Yup. But that is simple.
- Try writing the queries below in SQL.

The Movie Graph

Recommend

Let's recommend new co-actors for Tom Hanks. A basic recommendation approach is to find connections past an immediate neighborhood which are themselves well connected.

For Tom Hanks, that means:

1. Find actors that Tom Hanks hasn't yet worked with, but his co-actors have.
2. Find someone who can introduce Tom to his potential co-actor.

Extend Tom Hanks co-actors, to find co-co-actors who haven't work with Tom Hanks...

```
MATCH (tom:Person {name:"Tom Hanks"})-[:ACTED_IN]->(m)<-[:ACTED_IN]-(coActors),
      (coActors)-[:ACTED_IN]->(m2)<-[:ACTED_IN]-(cocoActors)
WHERE NOT (tom)-[:ACTED_IN]->(m2)
RETURN cocoActors.name AS Recommended, count(*) AS Strength ORDER BY Strength DESC
```

Find someone to introduce Tom Hanks to Tom Cruise

```
MATCH (tom:Person {name:"Tom Hanks"})-[:ACTED_IN]->(m)<-[:ACTED_IN]-(coActors),
      (coActors)-[:ACTED_IN]->(m2)<-[:ACTED_IN]-(cruise:Person {name:"Tom Cruise"})
RETURN tom, m, coActors, m2, cruise
```

Recommend

```
1 MATCH (tom:Person {name:"Tom Hanks"})-[:ACTED_IN]->(m)<-[:ACTED_IN]-(coActors),
2      (coActors)-[:ACTED_IN]->(m2)<-[:ACTED_IN]-(cocoActors)
3 WHERE NOT (tom)-[:ACTED_IN]->(m2)
4 RETURN cocoActors.name AS Recommended, count(*) AS Strength ORDER BY Strength DESC
```



\$ MATCH (tom:Person {name:"Tom Hanks"})-[:ACTED_IN]->(m)<-[:ACTED_IN]-(coActors), (coActors)-[:ACTED_IN]->(m2)<-[:ACTED_IN]-(cocoActors) ...



	Recommended	Strength
Rows	Tom Cruise	5
A	Zach Grenier	5
Text	Helen Hunt	4
</>	Cuba Gooding Jr.	4
Code	Keanu Reeves	4
	Tom Skerritt	3
	Carrie-Anne Moss	3
	Val Kilmer	3
	Bruno Kirby	3
	Philip Seymour Hoffman	3
	Billy Crystal	3
	Carrie Fisher	3

```

1 MATCH (tom:Person {name:"Tom Hanks"})-[:ACTED_IN]->(m)-[:ACTED_IN]-(coActors),
2     (coActors)-[:ACTED_IN]->(m2)-[:ACTED_IN]-(cruise:Person {name:"Tom Cruise"})
3 RETURN tom, m, coActors, m2, cruise

```

\$ MATCH (tom:Person {name:"Tom Hanks"})-[:ACTED_IN]->(m)-[:ACTED_IN]-(coActors), (coActors)-[:ACTED_IN]->(m2)-[:ACTED_IN]-(cruise:Person {name:"Tom Cruise"})

Graph

*(13) Movie(8) Person(5)

*(16) ACTED_IN(16)

Rows

Text

Code



Which actors have worked with both Tom Hanks and Tom Cruise?

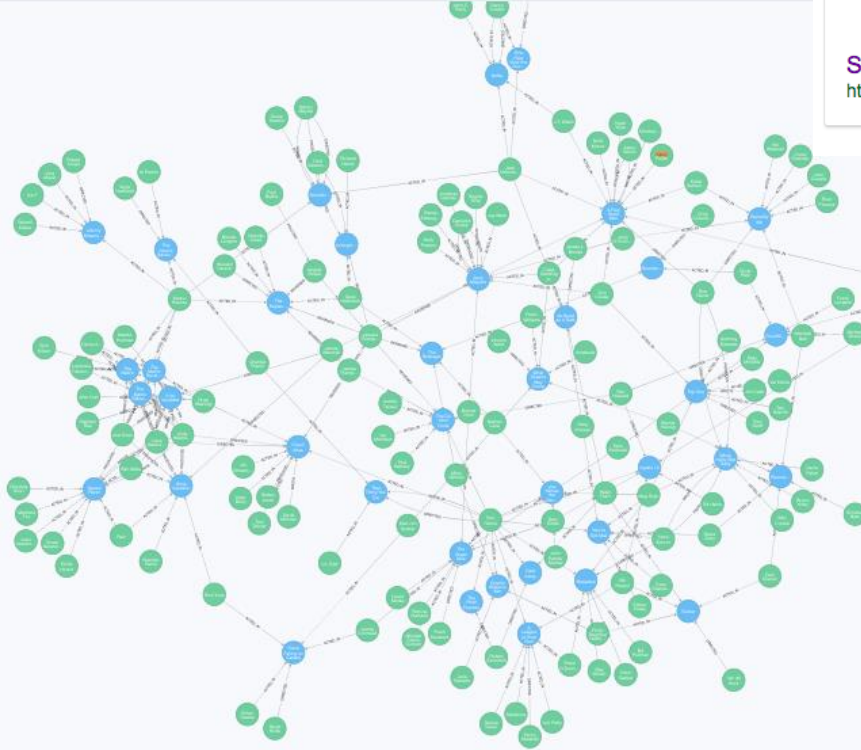
Displaying 13 nodes, 16 relationships (completed with 16 additional relationships).

AUTO-COMPLETE ☒

```
$ MATCH (s:Person { name: 'Kevin Bacon' })-[*0..6]-(m) return s,m
```

*(171) Movie(38) Person(133)

*(253) ACTED_IN(172) DIRECTED(44) FOLLOWS(3) PRODUCED(15) REVIEWED(9) WROTE(10)



Six Degrees of Kevin Bacon is a parlour game based on the "six degrees of separation" concept, which posits that any two people on Earth are six or fewer acquaintance links apart. Movie buffs challenge each other to find the shortest path between an arbitrary actor and prolific actor **Kevin Bacon**.



[Six Degrees of Kevin Bacon - Wikipedia](https://en.wikipedia.org/wiki/Six_Degrees_of_Kevin_Bacon)

https://en.wikipedia.org/wiki/Six_Degrees_of_Kevin_Bacon

[About this result](#)

[Feedback](#)

Six Degrees of Kevin Bacon

Game





How do you get from Kevin Bacon to Robert Longo?

